Especificación Técnica MVP: App de Enrutamiento (Zero-Budget)

Ingenieros a cargo: Juan Esteban Guerrero Vera, Santiago Casas Barreto, Jose Sebastian Villalba

1. Objetivo del Documento

Proveer una especificación técnica detallada y estructurada para ser utilizada como prompt de contexto en asistentes de desarrollo CLI (como Antigravity o Copilot). Este documento define la arquitectura, el modelo de datos y los endpoints requeridos para construir un MVP funcional a costo $0.

2. Stack Tecnológico (Capa Gratuita)

- **Frontend Móvil:** React Native (Expo).
- **Backend:** Python + FastAPI (Despliegue serverless recomendado en Vercel o Render).
- **Base de Datos:** Supabase (PostgreSQL). Ofrece la agilidad de Firebase pero manteniendo la estructura relacional.
- **Procesamiento OCR:** Google ML Kit Text Recognition v2 (Procesamiento on-device en React Native, 100% gratuito y offline).
- **Geocodificación:** Mapbox API.
- **Algoritmo de Optimización:** Google OR-Tools (Librería de Python en el backend).

3. Modelo de Datos Relacional (PostgreSQL / Supabase)

Tabla: drivers (Repartidores)

Campo	Tipo	Descripción
id	UUID	Primary Key
name	Varchar	Nombre del conductor

Campo	Tipo	Descripción
created_at	Timestamp	Fecha de creación

Tabla: routes (Rutas Diarias)

Campo	Tipo	Descripción
id	UUID	Primary Key
driver_id	UUID	Foreign Key → drivers.id
date	Date	Fecha de la ruta
status	Enum	'planning', 'in_progress', 'completed'

Tabla: stops (Paradas / Paquetes)

Campo	Tipo	Descripción
id	UUID	Primary Key
route_id	UUID	Foreign Key → routes.id
raw_address	Text	Dirección extraída por el OCR
normalized_address	Text	Dirección procesada
latitude	Float	Latitud geográfica
longitude	Float	Longitud geográfica
client_names	Text	Nombres de clientes (posibles duplicados)
packages_count	Int	Cantidad de paquetes en esta parada
sizes	JSONB	Ej: ["P", "M"]

Campo	Tipo	Descripción
van_locations	JSONB	Ej: ["Izquierda", "Fondo"]
optimized_order	Int	Orden sugerido por OR-Tools
status	Enum	'pending', 'delivered', 'failed'

4. Endpoints del Backend (FastAPI)

- **POST /api/v1/routes**: Crea una nueva ruta diaria para un conductor.
- **POST /api/v1/stops/batch-process**: Recibe un array de direcciones y metadatos escaneados desde la app.
    - **Lógica Interna**: Llama a Mapbox para obtener lat/lng. Si las coordenadas ya existen en la ruta actual, suma los metadatos (agrupa) en la parada existente.
- **POST /api/v1/routes/{route_id}/optimize**: Ejecuta OR-Tools.
    - **Lógica Interna**: Construye la matriz de distancias usando las coordenadas espaciales y devuelve el `optimized_order` para cada parada.
- **PUT /api/v1/stops/{stop_id}/reorder**: Permite al usuario reordenar manualmente una parada. Dispara un recálculo del resto de la ruta.

5. Lógica Crítica: Normalización de Direcciones Locales

El backend en FastAPI debe contar con una capa de limpieza (Regex/Parsing) antes de enviar la dirección a Mapbox. Las nomenclaturas en Bogotá pueden ser complejas, por lo que el script debe normalizar abreviaturas comunes extraídas por el OCR:

- "Cll", "Cl" → "Calle"
- "Cra", "Kr", "Krr" → "Carrera"
- "Tv", "Trv" → "Transversal"
- "Dg", "Diag" → "Diagonal"

- Asegurar que el OCR mantenga sufijos como "Sur" o "Este" para evitar coordenadas erróneas.

6. Flujo de Interfaz en React Native (Frontend)

1. **Vista de Cámara (Modo Lote):** ML Kit escanea el texto en tiempo real. Un rectángulo guía al usuario hacia la zona de la dirección.
2. **Modal Rápido:** Al detectar texto válido, la pantalla se congela y muestra botones enormes: [Pequeño] [Mediano] [Grande] y [Izquierda] [Derecha] [Fondo]. Al tocar, se guarda en el estado local y la cámara se reactiva inmediatamente.
3. **Sincronización:** Un botón "Procesar X Paquetes" envía el array completo al endpoint `/batch-process` de FastAPI.